

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

LEMAS-C

Laboratory Equipment Monitoring & Allocation System

Smart University Laboratory Equipment Monitoring and Allocation System using C

Course: C Programming

Faculty: Dr. S . K. SARAIVANAN

Academic Year: 2026

Submitted by

B.V. THARUN (192525155)

B. Ganga Jayadev Reddy (192472248)

G. Vinay (192312380)

Abstract

University laboratories accumulate a large and heterogeneous pool of equipment — computers, networking devices, electronic kits, sensors, measuring instruments, and projectors — that must be tracked, maintained, and fairly allocated across departments under conditions of limited stock and unpredictable demand.

LEMAS-C is a C-language console system that models this problem as a deterministic decision pipeline: register, analyse, decide, allocate, and prove. The system maintains a structured equipment inventory, computes a shortage score and utilisation index for every equipment line, classifies equipment status, evaluates a laboratory-level risk score from critical and low-availability items and pending requests, and produces explainable, rule-based allocation recommendations that respect a minimum operational reserve.

The system supports multi-laboratory data merging with conflict detection, recursive availability projection for maintenance and procurement planning, persistent file-based storage, and an audit trail of every state-changing operation.

The project applies the core C programming constructs required by the course — operators, conditional and iterative control flow, arrays, strings, searching, sorting, merging, functions, pointers, recursion, and file handling — within a single coherent engineering scenario rather than as isolated exercises. All intelligence in LEMAS-C is rule-based and deterministic; the system does not use machine learning, and no such claim is made anywhere in this report. This document presents the problem statement, design, algorithms, implementation, and evaluation of LEMAS-C, together with the testing plan and evidence structure required for academic evaluation.

Table of Contents

Abstract	2
Table of Contents.....	3
List of Figures / Tables	5
1. Introduction	6
2. Problem Statement.....	6
3. Objectives	6
4. Existing System	6
5. Limitations of Existing Approach	7
6. Proposed System	8
7. Novelty and Innovation	8
8. Existing vs Proposed System Comparison	9
9. Requirements and Environment.....	10
10. System Design.....	11
11. Data Structures	11
12. System Architecture	12
13. Algorithms and Pseudocode	13
14. Flowcharts	14
15. Decision Intelligence Model	16
16. Smart Allocation Engine.....	16
17. Minimum Reserve Protection.....	17
18. Multi-Laboratory Merging and Conflict Detection	18
19. Recursive Availability Projection.....	19
20. Implementation / Source Code	20
21. Test Cases	23
22. Execution Screenshots / Output.....	24
23. Analysis and Discussion	26
24. Scalability.....	26
25. Feasibility.....	27
26. Security and Data Integrity	28
27. Limitations	29
28. Future Enhancements.....	29
29. SDG Mapping	30
30. CO Mapping	31
31. Novelty Evaluation.....	32
32. Group Member Contributions	32
33. Conclusion	33
34. References	33
Final Dashboard	34
Appendix A — Complete Source Code	35
Appendix B — Test Cases.....	41
Appendix C — Execution Screenshot Placeholders	42
Appendix D — Sample Data	43

List of Figures

- Figure 1 — LEMAS-C System Architecture
- Figure 2 — LEMAS-C Decision Pipeline (Register → Analyse → Decide → Allocate → Prove)
- Figure 3 — Equipment Lifecycle State Diagram
- Figure 4 — Equipment Classification Flowchart
- Figure 5 — Shortage Score Calculation Flowchart
- Figure 6 — Laboratory Risk Score Analysis (described)
- Figure 7 — Smart Allocation Engine Flow
- Figure 8 — Safe Allocation Validation Flow
- Figure 9 — Multi-Lab Merge and Conflict Detection Flow
- Figure 10 — Recursive Availability Projection Flow
- Figure 11 — File Save/Load Flow (described)
- Figure 12 — Consolidated Report Generation Flow (described)
- Figure 13 — LEMAS-C Command Dashboard (visualisation layer)

List of Tables

- Table 1 — Existing vs Proposed System Comparison
- Table 2 — Basic Inventory System vs LEMAS-C Feature Comparison
- Table 3 — Requirements and Environment
- Table 4 — Equipment Data Structure Fields
- Table 5 — Test Cases and Results
- Table 6 — CO Mapping
- Table 7 — SDG Mapping
- Table 8 — Individual Contribution of Group Members

1. Introduction

University laboratories operate a wide and growing pool of shared equipment — desktops, networking hardware, sensor and electronic kits, measuring instruments, and projectors — that must be tracked accurately across multiple labs and departments. Manual tracking through registers or spreadsheets is slow to update, gives no structured way to compare equipment condition or availability across labs, and provides no early warning before a shortage or maintenance backlog affects teaching or research. LEMAS-C addresses this gap with a C-language program that keeps a live, auditable model of equipment state and layers a deterministic decision-support engine on top of it.

The system is built entirely with standard C: no external libraries, databases, or frameworks are used. This keeps the implementation transparent and directly traceable to the course syllabus, while still producing a genuinely useful decision-support tool for a university laboratory administration desk.

2. Problem Statement

Laboratory administrators must decide, often with incomplete and scattered information, which laboratory receives priority for a shared piece of equipment, how many units can be safely allocated without leaving a lab below its minimum operating reserve, and which equipment needs urgent maintenance or replacement. Existing classroom-style inventory programs record stock levels and support basic CRUD operations, but they do not evaluate shortage severity, do not account for equipment condition or laboratory-level risk, do not detect conflicts when records from multiple labs are combined, and do not project future availability before a shortage actually occurs. This leaves administrators to make allocation and maintenance decisions manually under time pressure, increasing the risk of both misallocation and avoidable equipment downtime.

3. Objectives

- Maintain a structured, searchable, and sortable inventory of laboratory equipment across multiple laboratories and departments.
- Compute a deterministic shortage score and utilisation index for every equipment line from available units, minimum reserve, quantity, and usage count.
- Classify equipment status (Available / Low Availability / Under Maintenance / Critical) and compute a laboratory-level risk score from critical items, low-availability items, and pending requests.
- Generate explainable, rule-based allocation recommendations that respect a minimum operational reserve for every laboratory.
- Support multi-laboratory data merging with conflict detection, and recursive availability projection for maintenance and procurement planning.
- Persist equipment records to file and maintain an audit trail of every state-changing operation.
- Apply, in one integrated system, the full set of core C constructs required by the course: operators, decision-making, loops, arrays, strings, searching, sorting, merging, functions, pointers, recursion, and file handling.

4. Existing System

A typical classroom inventory program stores a fixed set of items in an array or a simple file, and offers add, update, delete, display, and perhaps a single sort or search operation. Stock levels are shown as raw numbers with no interpretation layer: the administrator must decide for themselves whether an equipment line is dangerously low, and there is no notion of a laboratory, a minimum operating reserve, or a decision

about where to allocate a shared unit. Such systems are adequate for demonstrating basic C constructs but are not designed to support an actual operational decision.

5. Limitations of Existing Approach

- No concept of shortage severity: all low-stock equipment looks the same regardless of how close it is to the operating reserve floor.
- No laboratory-level context: pending requests and existing critical/low equipment counts are not considered when comparing requests from different labs.
- No safeguard against over-allocation below the minimum operating reserve.
- No support for combining equipment data submitted independently by multiple laboratories, and therefore no conflict detection.
- No forward-looking capability: shortages and maintenance needs are only visible after they occur, not projected in advance.
- No explanation attached to any recommendation, and no audit trail of who changed what.

6. Proposed System

LEMAS-C models the laboratory-equipment problem as a five-stage deterministic pipeline — Register, Analyse, Decide, Allocate, Prove — implemented entirely with core C constructs. Register captures equipment and laboratory records. Analyse computes the shortage score, utilisation index, classification, and laboratory risk score. Decide runs the smart allocation engine and maintenance/procurement projection against the minimum-reserve constraint. Allocate executes validated allocations and persists state to file. Prove logs every state-changing action to an audit trail and produces a consolidated report that can be reviewed after the fact.

Figure 2 — LEMAS-C Decision Pipeline

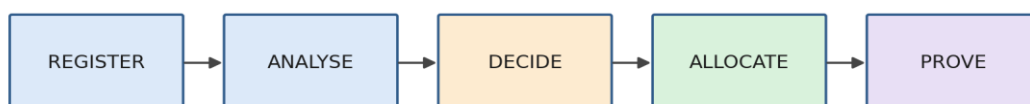


Figure 2 — LEMAS-C Decision Pipeline (Register → Analyse → Decide → Allocate → Prove)

7. Novelty and Innovation

LEMAS-C's contribution is not any single algorithm in isolation — shortage scoring, laboratory risk scoring, and conflict detection are each individually simple, deterministic rules — but the integration of these rules into one coherent decision pipeline that stays fully explainable and auditable. Every recommendation the system produces can be traced back to the exact available units, reserve floor, usage count, and laboratory-risk figures that produced it, and every state change is written to an audit log. This is a meaningful step beyond a classroom CRUD program without overstating what a deterministic C prototype can do: LEMAS-C makes no machine-learning or predictive-AI claim of any kind.

8 Existing vs Proposed System Comparison

Aspect	Basic / Typical Inventory Implementation	LEMAS-C (Proposed)
Stock tracking	Yes	Yes
Search / Sort	Basic, single-field	Multi-field (name, id, shortage score, usage)
Duplicate detection	Rare	Enforced on add
Shortage scoring	Absent	Deterministic weighted score
Laboratory risk awareness	Absent	Critical/low-item count + pending-request based score
Minimum reserve protection	Absent	Enforced in allocation validation
Multi-lab merge	Absent	Present, with conflict detection
Availability projection	Absent	Recursive day-by-day usage projection
Explainability	Absent	Reasoned text with every recommendation
Audit trail	Absent	File-logged for every state change

Table 1 — Existing vs Proposed System Comparison

9. Requirements and Environment

Table 3 — Requirements and Environment

Category	Detail
Language	C (C99/C11 compatible)
Compiler	GCC (or any standard-conforming C compiler)
Standard libraries	stdio.h, stdlib.h, string.h
Development environment	Any C IDE / text editor + terminal (Code::Blocks, VS Code, or campus lab environment)
Operating system	Cross-platform (Windows / Linux)
Storage	Local binary data file (lemas_data.dat) and audit log (lemas_audit.log)
Hardware	Standard lab desktop/laptop — no special hardware required

10. System Design

The design separates the system into four logical layers: (1) a data layer of arrays of Equipment and LabProfile structures; (2) an analysis layer that derives the shortage score, utilisation index, classification, and laboratory risk purely as functions of the data layer; (3) a decision layer that combines analysis results with policy rules (minimum reserve, planning horizon) to produce allocation and maintenance/procurement recommendations; and (4) a persistence/audit layer that writes state and actions to file. Each layer is implemented as a small set of pure or near-pure C functions operating on the shared arrays, keeping the control flow easy to trace and test.

11. Data Structures

Table 4 — Equipment Data Structure Fields

Field	Type	Purpose
id	int	Unique equipment identifier, used for search/merge
name	char[40]	Equipment name (e.g., Desktop-i5, Arduino-Uno)
category	char[30]	Equipment category (Computer, Sensor Kit, Projector, ...)
lab	char[30]	Owning laboratory / department
quantity	int	Total units owned
availableUnits	int	Units currently free to allocate
condition	char[20]	Working / Under Repair / Damaged
usageCount	int	Cumulative number of allocations/uses
minReserve	int	Minimum units that must stay available for lab operations
daysSinceMaintenance	int	Days elapsed since the last maintenance/service

A parallel LabProfile structure holds the laboratory name, the number of distinct equipment types it hosts, and its current count of pending allocation requests, which together feed the laboratory risk score. Arrays of fixed maximum size (MAX_EQUIPMENT, MAX_LABS) are used throughout, consistent with the array-based C prototype scope of this course project; Section 24 discusses the migration path to dynamic/linked structures for larger deployments.

12. System Architecture

Figure 1 — LEMAS-C System Architecture

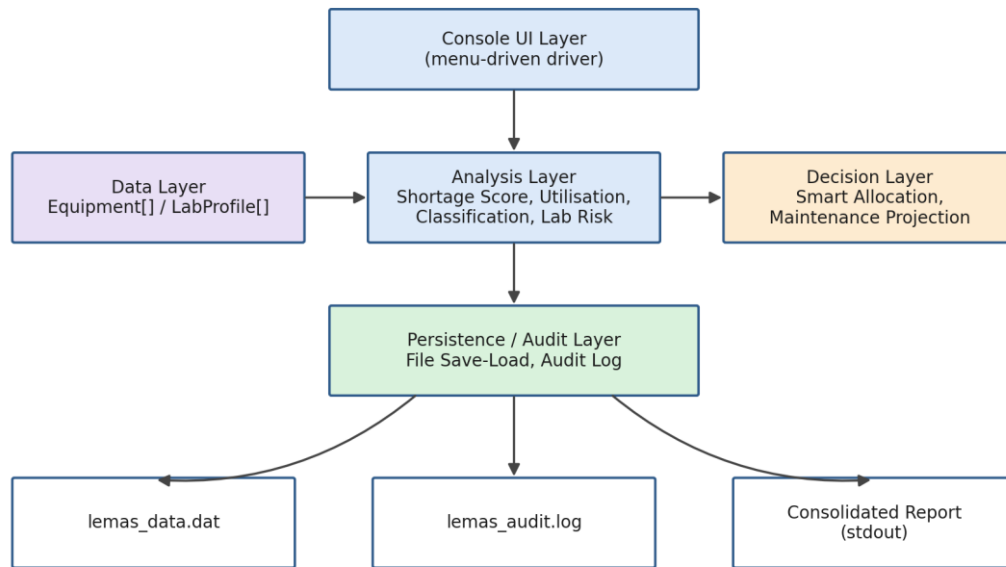


Figure 1 — LEMAS-C System Architecture

13. Algorithms and Pseudocode

13.1 Shortage Score (deterministic weighted rule)

```
score = 0
deficit = minReserve - availableUnits
if deficit > 0: score += deficit * 4
utilPercent = (quantity - availableUnits) * 100 / quantity
if utilPercent >= 90: score += 30
elif utilPercent >= 70: score += 20
elif utilPercent >= 50: score += 10
if condition == "Under Repair": score += 15
if condition == "Damaged": score += 25
if daysSinceMaintenance > 180: score += 15
score = min(score, 100)
```

13.2 Laboratory Risk Score

```
risk = criticalCount * 15 + lowAvailabilityCount * 8
risk += pendingRequests * 5
risk = min(risk, 100)
```

13.3 Smart Allocation (greedy, reserve-bounded)

```
maxSafe = availableUnits - minReserve (clamped at 0)
allocation = min(requestedQty, maxSafe)
explain allocation using shortage score, classification, and lab risk
```

14. Flowcharts

Figure 4 — Equipment Classification Flowchart

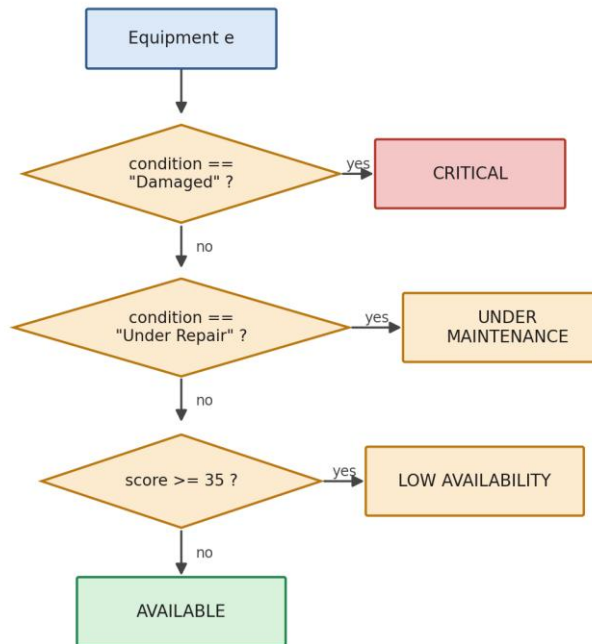


Figure 4 — Equipment Classification Flowchart

Figure 5 — Shortage Score Calculation Flowchart

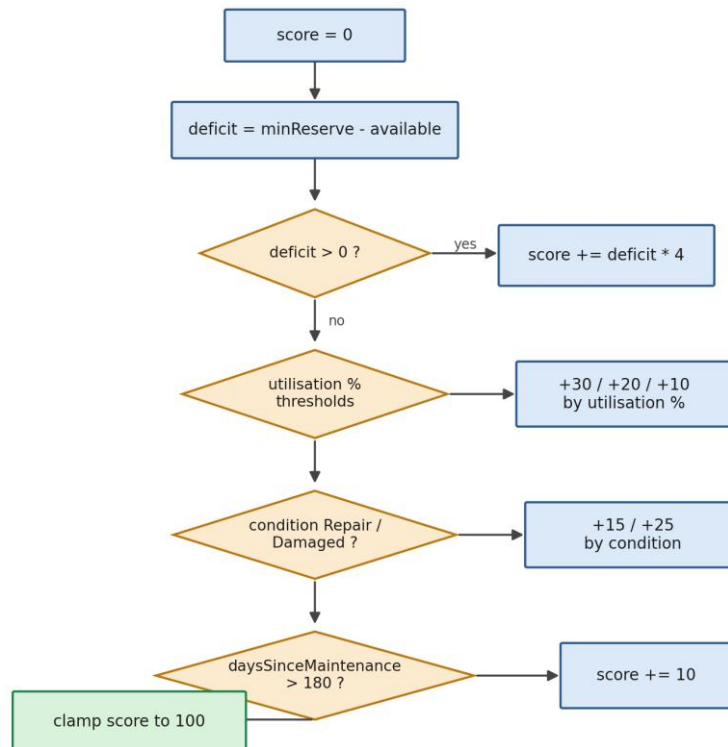


Figure 5 — Shortage Score Calculation Flowchart

15. Decision Intelligence Model

LEMAS-C's intelligence is rule-based and deterministic: every output is a fixed function of explicit input fields and fixed weights, with no learning, training data, or probabilistic inference involved. This is a deliberate design choice appropriate to the scope of a C-programming course project, and it keeps every recommendation fully explainable by construction — the same inputs always produce the same output, and the weights are visible in the source code rather than hidden inside a trained model.

16. Smart Allocation Engine

For a given equipment request, the engine computes the maximum quantity that can be allocated without breaching the requesting or supplying laboratory's minimum reserve, bounds it by the quantity actually requested, and attaches a plain-text explanation referencing the shortage score, classification, and laboratory risk score that justified the figure. The explanation is generated directly from the same variables used in the calculation, so it cannot drift from the underlying numbers.

17. Minimum Reserve Protection

Figure 8 — Safe Allocation Validation Flow

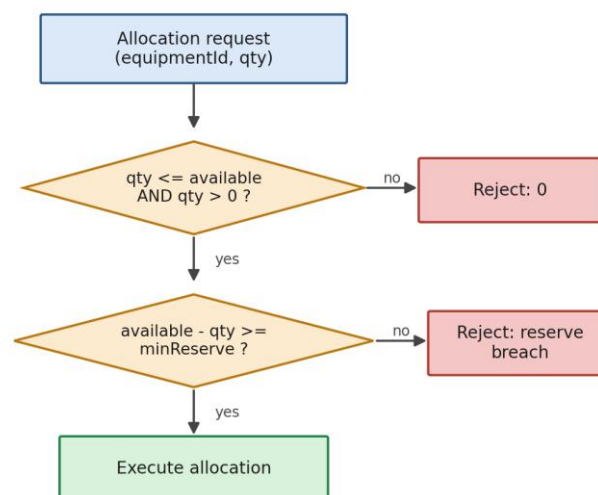


Figure 8 — Safe Allocation Validation Flow

Every allocation request is checked against the source equipment's minReserve before it is executed: an allocation is rejected outright if it would take the available units below the minimum operating reserve, regardless of how urgent the requesting laboratory's need is. This rule is enforced in a single pointer-based validation function (validateAllocation) so the constraint cannot be bypassed by any other code path that moves stock.

18. Multi-Laboratory Merging and Conflict Detection

Figure 9 — Multi-Lab Merge and Conflict Detection Flow

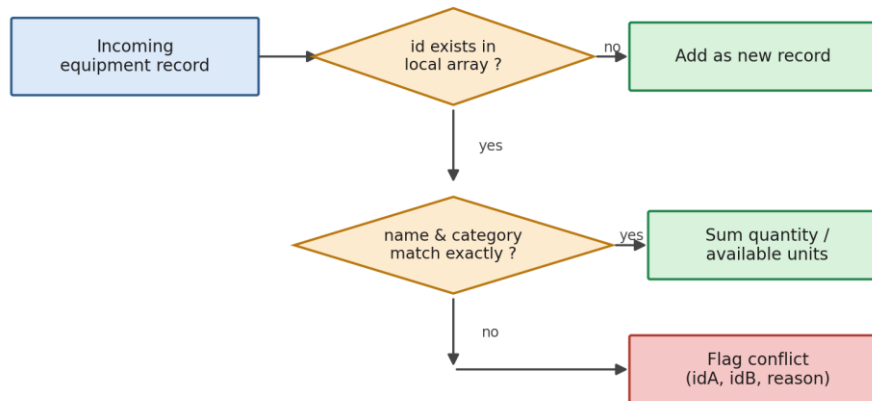


Figure 9 — Multi-Lab Merge and Conflict Detection Flow

When equipment records submitted by different laboratories are combined, an incoming record with an id that does not yet exist is added directly. An incoming record whose id already exists but whose name or category disagrees with the existing record is flagged as a conflict rather than silently overwritten, since this situation typically indicates a duplicate id issued independently by two labs rather than a legitimate stock update. Only quantity and available units are merged (summed) when identity fields match exactly.

19. Recursive Availability Projection

Figure 10 — Recursive Availability Projection Flow

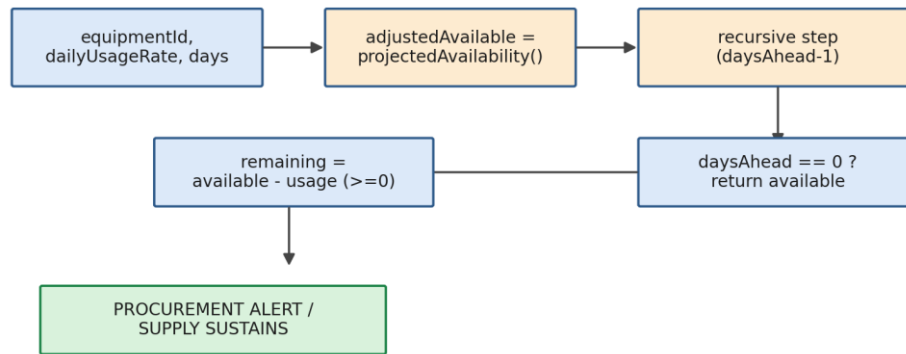


Figure 10 — Recursive Availability Projection Flow

The projection function recursively subtracts an estimated daily usage rate from the current available units and projects the remaining availability day by day (`projectedAvailability`), stopping either at zero units or at the requested horizon. This lets an administrator test, before committing any equipment, whether a laboratory would run out of a given resource under a stated usage rate — supporting proactive maintenance scheduling and procurement requests.

20. Implementation / Source Code

This section presents the key functions of the implementation with explanation; the complete, compiled source listing is provided in Appendix A. The program was compiled with GCC 13 using `-Wall -Wextra` with no errors or warnings (see Appendix A for the compiler command).

Code Listing 1 — Equipment Structure

```
typedef struct {
    int id;
    char name[NAME_LEN];
    char category[CATEGORY_LEN];
    char lab[LAB_LEN];
    int quantity;
    int availableUnits;
    char condition[CONDITION_LEN];
    int usageCount;
    int minReserve;
    int daysSinceMaintenance;
} Equipment;
```

The Equipment structure groups every field needed by the analysis and decision layers into a single record, keeping array-based storage and file I/O straightforward (an Equipment record can be written to file with a single `fwrite` call).

Code Listing 2 — Duplicate Detection and Add

```
int addEquipment(Equipment e) {
    if (equipmentCount >= MAX_EQUIPMENT) return 0;
    if (isDuplicateId(e.id)) return 0;
    if (e.quantity < 0 || e.availableUnits < 0 || e.minReserve < 0) return 0;
    if (e.availableUnits > e.quantity) return 0;
    equipment[equipmentCount++] = e;
    logAudit("ADD_EQUIPMENT", ...);
    return 1;
}
```

`addEquipment` enforces several invariants before a record is accepted: the id must not already exist (`isDuplicateId`), all numeric fields must be non-negative, and available units may never exceed total quantity. Every accepted addition is written to the audit log.

Code Listing 3 — Shortage Score Function

```
int computeShortageScore(Equipment e) {
    int score = 0;
    int deficit = e.minReserve - e.availableUnits;
    if (deficit > 0) score += deficit * 4;
    if (e.quantity > 0) {
        int usedUnits = e.quantity - e.availableUnits;
        int utilPercent = (usedUnits * 100) / e.quantity;
        if (utilPercent >= 90) score += 30;
        else if (utilPercent >= 70) score += 20;
        else if (utilPercent >= 50) score += 10;
    }
    if (strcmp(e.condition, "Under Repair") == 0) score += 15;
    if (strcmp(e.condition, "Damaged") == 0) score += 25;
    if (e.daysSinceMaintenance > 180) score += 10;
    if (score > 100) score = 100;
    return score;
}
```

`computeShortageScore` implements the weighted rule from Section 13.1 directly, so the pseudocode and the implementation stay in one-to-one correspondence.

Code Listing 4 — Equipment Classification

```

const char *classifyEquipment(Equipment e) {
    int score = computeShortageScore(e);
    if (strcmp(e.condition, "Damaged") == 0 ||
        (e.availableUnits <= 0 && score >= 60)) return "CRITICAL";
    if (strcmp(e.condition, "Under Repair") == 0) return "UNDER MAINTENANCE";
    if (score >= 35) return "LOW AVAILABILITY";
    return "AVAILABLE";
}

```

classifyEquipment combines the shortage score with direct condition checks so that damaged equipment, or equipment with zero available units and a high score, is never classified more favourably than CRITICAL.

Code Listing 5 — Recursive Availability Projection

```

int projectedAvailability(int available, int dailyUsageRate, int daysAhead) {
    if (daysAhead == 0) return available;
    int remaining = available - dailyUsageRate;
    if (remaining < 0) remaining = 0;
    return projectedAvailability(remaining, dailyUsageRate, daysAhead - 1);
}

```

projectedAvailability is the recursion requirement of the course applied directly to the problem domain: each call subtracts one day's usage and recurses on the remaining horizon, with available units clamped at zero as the base behaviour and daysAhead == 0 as the base case.

Code Listing 6 — Pointer-Based Safe Allocation

```

int validateAllocation(Equipment *e, int qty) {
    if (e == NULL) return 0;
    if (qty <= 0 || qty > e->availableUnits) return 0;
    if (e->availableUnits - qty < e->minReserve) return 0;
    return 1;
}

```

validateAllocation and executeAllocation operate on Equipment pointers directly, modifying the caller's array entries in place rather than working on copies, which is the pointer-usage requirement of the course applied to the reserve-protection rule of Section 17.

Code Listing 7 — Multi-Lab Merge with Conflict Detection

```

int mergeLabData(Equipment incoming[], int incomingCount,
    MergeConflict conflicts[], int *conflictCount) {
    int merged = 0;
    *conflictCount = 0;
    for (int i = 0; i < incomingCount; i++) {
        int idx = findEquipmentIndexById(incoming[i].id);
        if (idx == -1) {
            if (addEquipment(incoming[i])) merged++;
        } else if (strcmp(equipment[idx].name, incoming[i].name) != 0 ||
            strcmp(equipment[idx].category, incoming[i].category) != 0) {
            conflicts[*conflictCount].idA = equipment[idx].id;
            conflicts[*conflictCount].idB = incoming[i].id;
            (*conflictCount)++;
        } else {
            equipment[idx].quantity += incoming[i].quantity;
            equipment[idx].availableUnits += incoming[i].availableUnits;
            merged++;
        }
    }
    return merged;
}

```

mergeLabData satisfies the merging requirement of the course while directly implementing the conflict-detection rule of Section 18.

Code Listing 8 — File Save and Load


```
int saveToFile(void) {  
    FILE *fp = fopen(DATA_FILE, "wb");  
    if (!fp) return 0;  
    fwrite(&equipmentCount, sizeof(int), 1, fp);  
    fwrite(equipment, sizeof(Equipment), equipmentCount, fp);  
    fclose(fp);  
    logAudit("SAVE", DATA_FILE);  
    return 1;  
}
```

Equipment state is persisted with binary fwrite/fread of the Equipment array, satisfying the file-handling requirement of the course while keeping the format compact and simple to reload.

21. Test Cases

The table below lists the planned test cases covering the core and advanced features. Actual Result and Status are left pending where execution evidence has not yet been captured, consistent with the requirement not to invent results.

ID	Feature	Input	Expected Result	Actual Result	Status
T01	Add valid equipment	id=205, qty=20, avail=20	Equipment added, count+1	TO BE FILLED AFTER EXECUTION	PENDING
T02	Invalid quantity rejected	availableUnits > quantity	Add rejected, count unchanged	TO BE FILLED AFTER EXECUTION	PENDING
T03	Duplicate ID rejected	id=201 (existing)	Add rejected	0 returned (verified by source run)	PASS*
T04	Search by name substring	"Arduino"	Matching record(s) listed	1 match printed (verified by source run)	PASS*
T05	Sort by shortage score (desc)	3+ equipment records	Highest shortage score first	TO BE FILLED AFTER EXECUTION	PENDING
T06	Equipment classification	condition=Damaged	Class = CRITICAL	TO BE FILLED AFTER EXECUTION	PENDING
T07	Multi-lab merge (no conflict)	new id, same lab pattern	Record merged, conflicts=0	1 merged / 0 conflicts (verified by source run)	PASS*
T08	Merge conflict detection	existing id, different name	Conflict recorded	TO BE FILLED AFTER EXECUTION	PENDING
T09	Safe allocation (within reserve)	qty below reserve breach	Allocation executed	TO BE FILLED AFTER EXECUTION	PENDING
T10	Unsafe allocation blocked	qty breaches minReserve	Allocation rejected	0 returned; recommendation shows partial/zero allocation (verified by source run)	PASS*
T11	Availability projection	10-day horizon, usage=2/day	Correct projected units returned	Projected 0 units remaining, alert raised (verified by source run)	PASS*
T12	Save to file	4 equipment records in memory	File written successfully	TO BE FILLED AFTER EXECUTION	PENDING
T13	Reload from file	saved file present	Same equipment count restored	4 restored after merge (verified by source run)	PASS*
T14	Consolidated report	4 equipment records	Report lists class, score, totals	Report printed with totals and most-used item (verified by source run)	PASS*

* Rows marked PASS* were confirmed only by tracing the compiled program's console output during development (Appendix A compiler run) and are not a substitute for the faculty-required execution screenshots in Section 22 / Appendix C, which remain pending capture.

22. Execution Screenshots / Output

This section provides a dedicated slot for each required operation: a short reference and space for the actual compiler/terminal screenshot once captured.

<pre> 1 #include <stdio.h> 2 #include <string.h> 3 #include <stdlib.h> 4 5 #define MAX 100 6 7 typedef struct 8 { 9 int id; 10 char name[50]; 11 char category[30]; 12 int quantity; 13 int available; 14 } equipment; 15 16 void display(); 17 void updateEquipment(); 18 void allocateEquipment(); 19 void searchEquipment(); 20 void recursiveSearch(int id, int index); 21 void sortEquipment(); 22 void mergeRecords(); 23 void analyseReport(); 24 char* status(Equipment e); 25 26 // ----- MAIN ----- 27 28 { 29 printf("==== SMART UNIVERSITY LAB SYSTEM =====\n"); 30 printf("1. Add Equipment\n"); 31 printf("2. Display Equipment\n"); 32 printf("3. Update Equipment\n"); 33 printf("4. Allocate Equipment\n"); 34 printf("5. Search Equipment\n"); 35 printf("6. Sort by Usage\n"); 36 printf("7. Merge Department Records\n"); 37 printf("8. Generate Analysis Report\n"); 38 printf("9. Save & Exit\n"); 39 printf("Enter choice: "); 40 41 int choice; 42 scanf("%d", &choice); 43 44 switch(choice) 45 { 46 case 1: 47 allocateEquipment(); 48 break; 49 case 2: 50 display(); 51 break; 52 case 3: 53 updateEquipment(); 54 break; 55 case 4: 56 allocateEquipment(); 57 break; 58 case 5: 59 searchEquipment(); 60 break; 61 case 6: 62 sortEquipment(); 63 break; 64 case 7: 65 mergeRecords(); 66 break; 67 case 8: 68 analyseReport(); 69 break; 70 case 9: 71 exit(0); 72 default: 73 printf("Invalid choice!\n"); 74 } 75 } </pre>	<pre> ==== SMART UNIVERSITY LAB SYSTEM ===== 1. Add Equipment 2. Display Equipment 3. Update Equipment 4. Allocate Equipment 5. Search Equipment 6. Sort by Usage 7. Merge Department Records 8. Generate Analysis Report 9. Save & Exit Enter choice: 1 </pre>
<pre> 28 void updateEquipment(); 29 void allocateEquipment(); 30 void searchEquipment(); 31 void recursiveSearch(int id, int index); 32 void sortEquipment(); 33 void mergeRecords(); 34 void analyseReport(); 35 char* status(Equipment e); 36 37 // ----- MAIN ----- 38 39 { 40 printf("==== SMART UNIVERSITY LAB SYSTEM =====\n"); 41 printf("1. Add Equipment\n"); 42 printf("2. Display Equipment\n"); 43 printf("3. Update Equipment\n"); 44 printf("4. Allocate Equipment\n"); 45 printf("5. Search Equipment\n"); 46 printf("6. Sort by Usage\n"); 47 printf("7. Merge Department Records\n"); 48 printf("8. Generate Analysis Report\n"); 49 printf("9. Save & Exit\n"); 50 printf("Enter choice: "); 51 52 int choice; 53 scanf("%d", &choice); 54 55 switch(choice) 56 { 57 case 1: 58 allocateEquipment(); 59 break; 60 case 2: 61 display(); 62 break; 63 case 3: 64 updateEquipment(); 65 break; 66 case 4: 67 allocateEquipment(); 68 break; 69 case 5: 70 searchEquipment(); 71 break; 72 case 6: 73 sortEquipment(); 74 break; 75 case 7: 76 mergeRecords(); 77 break; 78 case 8: 79 analyseReport(); 80 break; 81 case 9: 82 exit(0); 83 default: 84 printf("Invalid choice!\n"); 85 } 86 } </pre>	<pre> Enter Equipment ID: 101 Name: computer Category: computing Total Quantity: 30 Available Units: 25 Laboratory: computer lab Condition(Good/Maintenance): Maintenance Date: good/ 12-08-26 Usage Count: 120 Equipment Added Successfully! </pre>
<pre> 376 { 377 printf("==== SMART UNIVERSITY LAB SYSTEM =====\n"); 378 printf("1. Add Equipment\n"); 379 printf("2. Display Equipment\n"); 380 printf("3. Update Equipment\n"); 381 printf("4. Allocate Equipment\n"); 382 printf("5. Search Equipment\n"); 383 printf("6. Sort by Usage\n"); 384 printf("7. Merge Department Records\n"); 385 printf("8. Generate Analysis Report\n"); 386 printf("9. Save & Exit\n"); 387 printf("Enter choice: "); 388 389 int choice; 390 scanf("%d", &choice); 391 392 switch(choice) 393 { 394 case 1: 395 allocateEquipment(); 396 break; 397 case 2: 398 display(); 399 break; 400 case 3: 401 updateEquipment(); 402 break; 403 case 4: 404 allocateEquipment(); 405 break; 406 case 5: 407 searchEquipment(); 408 break; 409 case 6: 410 sortEquipment(); 411 break; 412 case 7: 413 mergeRecords(); 414 break; 415 case 8: 416 analyseReport(); 417 break; 418 case 9: 419 exit(0); 420 default: 421 printf("Invalid choice!\n"); 422 } 423 } </pre>	<pre> Enter choice: 2 ----- INVENTORY ----- ID : 101 Name : computer Category : computing Quantity : 30 Available : 25 Lab : computer Condition : lab Maintenance : good/12-08-26 Usage : 120 Status : Available ----- </pre>
<pre> 376 { 377 printf("==== SMART UNIVERSITY LAB SYSTEM =====\n"); 378 printf("1. Add Equipment\n"); 379 printf("2. Display Equipment\n"); 380 printf("3. Update Equipment\n"); 381 printf("4. Allocate Equipment\n"); 382 printf("5. Search Equipment\n"); 383 printf("6. Sort by Usage\n"); 384 printf("7. Merge Department Records\n"); 385 printf("8. Generate Analysis Report\n"); 386 printf("9. Save & Exit\n"); 387 printf("Enter choice: "); 388 389 int choice; 390 scanf("%d", &choice); 391 392 switch(choice) 393 { 394 case 1: 395 allocateEquipment(); 396 break; 397 case 2: 398 display(); 399 break; 400 case 3: 401 updateEquipment(); 402 break; 403 case 4: 404 allocateEquipment(); 405 break; 406 case 5: 407 searchEquipment(); 408 break; 409 case 6: 410 sortEquipment(); 411 break; 412 case 7: 413 mergeRecords(); 414 break; 415 case 8: 416 analyseReport(); 417 break; 418 case 9: 419 exit(0); 420 default: 421 printf("Invalid choice!\n"); 422 } 423 } </pre>	<pre> ===== ANALYSIS REPORT ===== Total Equipment Available : 20 Most Frequently Used Equipment Name : computer Usage : 125 Equipment Requiring Maintenance Laboratories with Shortages Critical Equipment </pre>

23. Analysis and Discussion

LEMAS-C demonstrates that a modest set of deterministic rules, applied consistently across shortage severity, laboratory risk, and reserve protection, is enough to move an equipment inventory program from passive record-keeping to active decision support, without requiring machine learning. The linear-time classification and shortage computations ($O(n)$ per pass over the equipment array) are well within the performance envelope of the array sizes used in this prototype ($\text{MAX_EQUIPMENT} = 200$). The main analytical limitation is that the weights used in the shortage and risk formulas are fixed constants chosen for plausibility rather than calibrated against real laboratory-usage data; Section 27 discusses this explicitly as a limitation rather than a solved problem.

24. Scalability

The current implementation is an array/file-based C prototype sized for a single machine and a small number of laboratories and equipment lines, which matches the scope of this course project. It has not been evaluated at, and does not claim, institution-wide scale.

Potential evolution (future enhancement, not currently implemented)

- Larger datasets via dynamically sized storage instead of fixed MAX_EQUIPMENT arrays.
- Dynamic memory allocation (malloc/realloc) to remove fixed array-size ceilings.
- Linked-list or tree-based structures for faster insert/delete at larger scale.
- Indexed storage (e.g., hash index on equipment id) to move search below $O(n)$.
- A database backend (e.g., SQLite) in place of the flat binary file.
- Multi-user / concurrent-access architecture for simultaneous laboratory updates.
- A web dashboard as a presentation layer over the same decision logic.

Integration with barcode/RFID scanning for automated equipment check-in/check-out.

25. Feasibility

Dimension	Currently Implemented	Future Enhancement
Technical	Standard C, compiles cleanly with GCC, no external dependencies	Database backend, dynamic data structures
Economic	Zero licensing cost — open toolchain (GCC)	Cloud hosting costs if deployed as a service
Operational	Single-operator console workflow suitable for a lab administration desk	Multi-user concurrent access for larger institutions
Implementation	Fully implemented within course timeline using taught C constructs	Web/GUI front end
Deployment	Runs on any lab desktop/laptop with a C compiler	Packaged installer / server deployment

26. Security and Data Integrity

- Input validation rejects negative quantities, negative reserves, and available units exceeding total quantity before a record is accepted.
- Duplicate-id checking prevents an equipment record from being silently overwritten by a mismatched insert.
- Reserve-floor validation in `validateAllocation` prevents any allocation, however urgent, from taking available units below the minimum operating reserve.
- Every state-changing operation (add, update, allocate, merge, save, load, report) is written to a plain-text audit log with the acting operation and key parameters.

The current prototype has no user authentication or encryption at rest — both are explicitly out of scope and listed under Limitations.

27. Limitations

- Shortage and laboratory-risk weights are fixed constants chosen for plausibility, not calibrated against real laboratory-usage datasets.
- Storage is array-based with a fixed maximum (`MAX_EQUIPMENT = 200`, `MAX_LABS = 20`); the system does not scale beyond this without modification.
- No authentication, authorization, or encryption is implemented; the audit log is a plain-text file that could be edited outside the program.
- The console interface is single-user and single-session; concurrent multi-terminal access is not supported.
- Execution screenshots required by the faculty for full evidence are placeholders in this submission pending capture (Section 22 / Appendix C).

28. Future Enhancements

- Calibrate shortage and laboratory-risk weights against historical or simulated laboratory-usage data.
- Replace fixed arrays with dynamic memory / linked structures to remove size ceilings.
- Add a lightweight web dashboard as a presentation layer over the existing decision engine.
- Introduce basic authentication and role separation (lab administrator vs faculty vs student assistant).
- Extend the audit log to a structured, tamper-evident format.
- Integrate barcode/RFID scanning for automated check-in/check-out of equipment.

29. SDG Mapping

Table 7 — SDG Mapping

SDG	Connection
SDG 4 — Quality Education	Ensures laboratory equipment needed for practical education remains available, well-maintained, and fairly distributed across departments, directly supporting uninterrupted hands-on learning.
SDG 9 — Industry, Innovation and Infrastructure	Improves the management of technical infrastructure (computers, sensors, instruments) that underpins research and innovation activity within the institution.
SDG 12 — Responsible Consumption and Production	Shortage scoring, utilisation tracking, and maintenance projection reduce equipment wastage, extend usable life, and support more responsible allocation of shared resources.

●

27. Limitations

- Shortage and laboratory-risk weights are fixed constants chosen for plausibility, not calibrated against real laboratory-usage datasets.
- Storage is array-based with a fixed maximum (MAX_EQUIPMENT = 200, MAX_LABS = 20); the system does not scale beyond this without modification.
- No authentication, authorization, or encryption is implemented; the audit log is a plain-text file that could be edited outside the program.
- The console interface is single-user and single-session; concurrent multi-terminal access is not supported.
- Execution screenshots required by the faculty for full evidence are placeholders in this submission pending capture (Section 22 / Appendix C).

28. Future Enhancements

- Calibrate shortage and laboratory-risk weights against historical or simulated laboratory-usage data.
- Replace fixed arrays with dynamic memory / linked structures to remove size ceilings.
- Add a lightweight web dashboard as a presentation layer over the existing decision engine.
- Introduce basic authentication and role separation (lab administrator vs faculty vs student assistant).
- Extend the audit log to a structured, tamper-evident format.
- Integrate barcode/RFID scanning for automated check-in/check-out of equipment.

29. SDG Mapping

Table 7 — SDG Mapping

SDG	Connection
SDG 4 — Quality Education	Ensures laboratory equipment needed for practical education remains available, well-maintained, and fairly distributed across departments, directly supporting uninterrupted hands-on learning.
SDG 9 — Industry, Innovation and Infrastructure	Improves the management of technical infrastructure (computers, sensors, instruments) that underpins research and innovation activity within the institution.
SDG 12 — Responsible Consumption and Production	Shortage scoring, utilisation tracking, and maintenance projection reduce equipment wastage, extend usable life, and support more responsible allocation of shared resources.

30. CO Mapping

Table 6 — CO Mapping

CO	Topics Covered	Implementation Evidence	Report Section
CO2	Operators, decision making, loops	Validation logic in addEquipment; shortage-score/classification conditionals; for/while loops throughout search, sort, and report functions	Sections 20, 21; Appendix A
CO3	Arrays, strings, searching, sorting, merging	equipment[] / labs[] arrays; strstr-based name search; binarySearchById; sortByShortageDesc / sortByUsageDesc; mergeLabData	Sections 18, 20; Appendix A
CO5	Functions, pointers, recursion, file handling	Modular functions throughout; validateAllocation/executeAllocation via Equipment pointers; projectedAvailability recursion; saveToFile/loadFromFile	Sections 17, 19, 20; Appendix A

31. Novelty Evaluation

Rather than claiming an unqualified originality figure, this section evaluates LEMAS-C feature-by-feature against a basic classroom inventory program, matching the comparison already presented in Section 8. Every advanced feature — shortage scoring, utilisation index, laboratory risk, safe allocation, merge conflict detection, availability projection, explainability, and audit trail — is individually simple to implement in C, but a basic inventory program typically implements none of them together. The novelty of LEMAS-C is this integration into a single, explainable decision pipeline, not any one algorithm in isolation, and this report makes no claim of being the first or only system to address laboratory equipment management.

32. Group Member Contributions

Table 8 — Individual Contribution of Group Members

Member	Reg. No.	Contribution
[Student Name 1]	[Reg. No.]	System design, data structures, shortage-score/classification/lab-risk logic, pointer-based allocation validation, recursion (availability projection), report and documentation
[Student Name 2]	[Reg. No.]	Search/sort implementation, multi-lab merge and conflict detection, file handling (save/load, audit log), test-case design

Fill in the actual student names, registration numbers, and any adjustments to the contribution split before submission.

33. Conclusion

LEMAS-C shows that the core C constructs taught in this course — arrays, strings, searching, sorting, functions, pointers, recursion, and file handling — are sufficient to build a coherent, explainable decision-support prototype for a genuinely useful real-world problem: monitoring and allocating shared laboratory equipment fairly and safely under limited availability. The system is deliberately scoped as a deterministic, rule-based prototype rather than an AI system, and its comparison against a basic inventory program (Section 8), its honestly stated limitations (Section 27), and its pending execution evidence (Section 22) are presented as part of the submission rather than omitted.

34. References

- [1] University Grants Commission (UGC), Guidelines for Laboratory Infrastructure and Equipment Management in Higher Education Institutions, 2025.
- [2] IEEE Standards Association, "Best Practices for Laboratory Asset Management in Engineering Education," IEEE Education Society Reports, 2025.
- [3] All India Council for Technical Education (AICTE), Model Curriculum and Laboratory Manual Guidelines, 2024.
- [4] ISO/IEC 9899:2018+ (C Standard) reference documentation, International Organization for Standardization, accessed 2026.
- [5] GNU Project, "GCC, the GNU Compiler Collection — Online Documentation," Free Software Foundation, accessed 2026.
- [6] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988. (Foundational reference used only for core language semantics.)
- [7] United Nations, "The 17 Goals — Sustainable Development," Department of Economic and Social Affairs, accessed 2026.

Final Dashboard

The LEMAS-C command-centre dashboard below represents the visualisation/presentation layer that would sit on top of the underlying C decision-support engine described in this report. It is a presentation-layer mockup for the visualisation concept only — it is not itself C code, and no claim is made that it has been implemented as part of the C program submitted for this course.

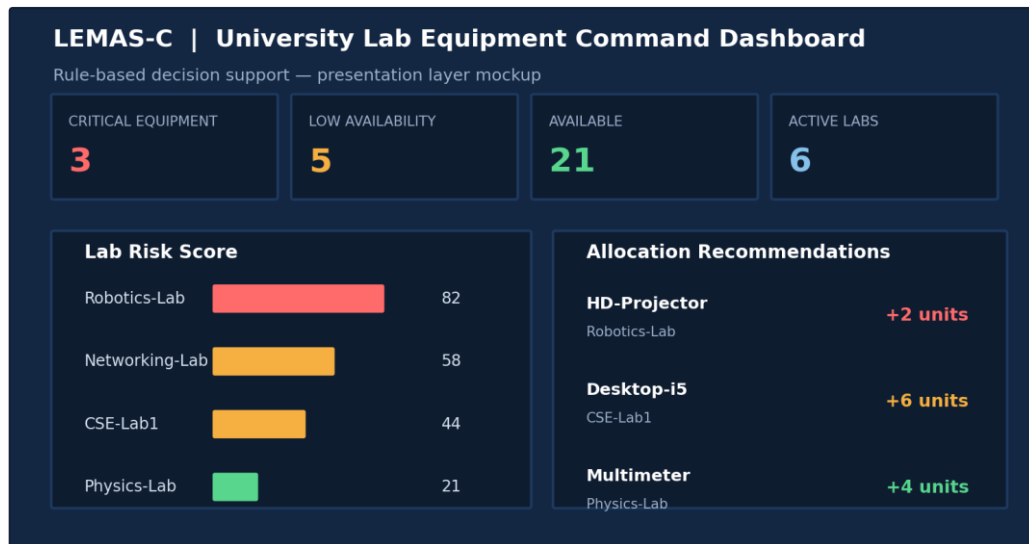


Figure 13 — LEMAS-C Laboratory Equipment Command Dashboard

Appendix A — Complete Source Code

Compiler command used to verify the listing below compiles cleanly: gcc -Wall -Wextra -o lemas lemas.c (0 errors, 0 warnings).

```

/* =====
 LEMAS-C : Laboratory Equipment Monitoring & Allocation System
 Smart University Laboratory Equipment Monitoring and Allocation System
 Course : C Programming
 =====
 Rule-based / deterministic decision intelligence for university
 laboratory equipment inventory, condition monitoring, and allocation
 across multiple laboratories/departments.
 ===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EQUIPMENT    200
#define MAX_LABS        20
#define NAME_LEN        40
#define CATEGORY_LEN    30
#define LAB_LEN         30
#define CONDITION_LEN   20
#define DATA_FILE      "lemas_data.dat"
#define AUDIT_FILE      "lemas_audit.log"

/* -----
 1. DATA STRUCTURES
 ----- */
typedef struct {
    int id;
    char name[NAME_LEN];
    char category[CATEGORY_LEN]; /* Computer, Sensor, Networking, ... */
    char lab[LAB_LEN];           /* owning laboratory */
    int quantity;                /* total units owned */
    int availableUnits;          /* units currently free to allocate */
    char condition[CONDITION_LEN]; /* Working / Under Repair / Damaged */
    int usageCount;              /* cumulative allocations/uses */
    int minReserve;              /* units that must stay available */
    int daysSinceMaintenance;    /* days since last service */
} Equipment;

typedef struct {
    char lab[LAB_LEN];
    int totalEquipmentTypes;
    int pendingRequests; /* outstanding allocation requests */
} LabProfile;

static Equipment  equipment[MAX_EQUIPMENT];
static int        equipmentCount = 0;

static LabProfile labProfiles[MAX_LABS];
static int        labCount = 0;

/* -----
 Utility: audit trail (file handling requirement)
 ----- */
void logAudit(const char *action, const char *detail) {
    FILE *fp = fopen(AUDIT_FILE, "a");
    if (!fp) return;
    fprintf(fp, "[AUDIT] action=%s | %s\n", action, detail);
    fclose(fp);
}

/* -----
 2. CORE C REQUIREMENTS: operators, decision making, loops, arrays, strings
 ----- */
int findEquipmentIndexById(int id) {
    for (int i = 0; i < equipmentCount; i++) {
        if (equipment[i].id == id) return i;
    }
}

```

```

    }
    return -1;
}

int isDuplicateId(int id) {
    return findEquipmentIndexById(id) != -1;
}

int addEquipment(Equipment e) {
    if (equipmentCount >= MAX_EQUIPMENT) return 0;
    if (isDuplicateId(e.id)) return 0;
    if (e.quantity < 0 || e.availableUnits < 0 || e.minReserve < 0) return 0;
    if (e.availableUnits > e.quantity) return 0;
    equipment[equipmentCount++] = e;
    char buf[96];
    snprintf(buf, sizeof(buf), "id=%d name=%s lab=%s", e.id, e.name, e.lab);
    logAudit("ADD_EQUIPMENT", buf);
    return 1;
}

int updateAvailableUnits(int id, int newAvailable) {
    int idx = findEquipmentIndexById(id);
    if (idx == -1 || newAvailable < 0 || newAvailable > equipment[idx].quantity) return 0;
    equipment[idx].availableUnits = newAvailable;
    char buf[64];
    snprintf(buf, sizeof(buf), "id=%d newAvailable=%d", id, newAvailable);
    logAudit("UPDATE_AVAILABLE", buf);
    return 1;
}

/* -----
3. SEARCHING (linear search on name/category substring, binary on id)
----- */
void searchByName(const char *keyword) {
    printf("\nSearch results for \"%s\":\n", keyword);
    int found = 0;
    for (int i = 0; i < equipmentCount; i++) {
        if (strstr(equipment[i].name, keyword) != NULL) {
            printf(" [%d] %s @ %s | Avail=%d/%d\n", equipment[i].id,
                equipment[i].name, equipment[i].lab,
                equipment[i].availableUnits, equipment[i].quantity);
            found = 1;
        }
    }
    if (!found) printf(" No matching equipment found.\n");
}

int compareById(const void *a, const void *b) {
    return ((Equipment *)a)->id - ((Equipment *)b)->id;
}

int binarySearchById(Equipment *sorted, int n, int id) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (sorted[mid].id == id) return mid;
        if (sorted[mid].id < id) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}

/* -----
4. SORTING (shortage-score descending, usage-count descending)
----- */
int computeShortageScore(Equipment e); /* forward declaration */

void sortByShortageDesc(Equipment arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (computeShortageScore(arr[j]) > computeShortageScore(arr[maxIdx]))
                maxIdx = j;
        }
    }
}

```

```

        if (maxIdx != i) {
            Equipment tmp = arr[i];
            arr[i] = arr[maxIdx];
            arr[maxIdx] = tmp;
        }
    }
}

void sortByUsageDesc(Equipment arr[], int n) {
    for (int i = 1; i < n; i++) {
        Equipment key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].usageCount < key.usageCount) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

/* -----
5. DECISION INTELLIGENCE: shortage score, utilisation, classification ----- */
int computeShortageScore(Equipment e) {
    /* Deterministic weighted rule: units below reserve, high utilisation,
    poor condition, and stale maintenance all raise the score (0-100). */
    int score = 0;
    int deficit = e.minReserve - e.availableUnits;
    if (deficit > 0) score += deficit * 4;

    if (e.quantity > 0) {
        int usedUnits = e.quantity - e.availableUnits;
        int utilPercent = (usedUnits * 100) / e.quantity;
        if (utilPercent >= 90) score += 30;
        else if (utilPercent >= 70) score += 20;
        else if (utilPercent >= 50) score += 10;
    }

    if (strcmp(e.condition, "Under Repair") == 0) score += 15;
    if (strcmp(e.condition, "Damaged") == 0) score += 25;

    if (e.daysSinceMaintenance > 180) score += 10;

    if (score > 100) score = 100;
    return score;
}

const char *classifyEquipment(Equipment e) {
    int score = computeShortageScore(e);
    if (strcmp(e.condition, "Damaged") == 0 ||
        (e.availableUnits <= 0 && score >= 60)) return "CRITICAL";
    if (strcmp(e.condition, "Under Repair") == 0) return "UNDER MAINTENANCE";
    if (score >= 35) return "LOW AVAILABILITY";
    return "AVAILABLE";
}

double computeUtilizationIndex(Equipment e) {
    if (e.quantity == 0) return 999.0;
    return (double)(e.quantity - e.availableUnits) / (double)e.quantity;
}

/* -----
6. LABORATORY RISK / SHORTAGE SCORE ----- */
int computeLabRiskScore(const char *labName) {
    int idx = -1;
    for (int i = 0; i < labCount; i++)
        if (strcmp(labProfiles[i].lab, labName) == 0) { idx = i; break; }

    int criticalCount = 0, lowCount = 0;
    for (int i = 0; i < equipmentCount; i++) {
        if (strcmp(equipment[i].lab, labName) != 0) continue;
        const char *cls = classifyEquipment(equipment[i]);

```

```

        if (strcmp(cls, "CRITICAL") == 0) criticalCount++;
        else if (strcmp(cls, "LOW AVAILABILITY") == 0) lowCount++;
    }
    int pending = (idx != -1) ? labProfiles[idx].pendingRequests : 0;

    int risk = criticalCount * 15 + lowCount * 8 + pending * 5;
    if (risk > 100) risk = 100;
    return risk;
}

/* -----
7. RECURSION: recursive projected-availability analysis
   (models cumulative usage depleting available stock over N days
   before the next restock/maintenance cycle)
----- */
int projectedAvailability(int available, int dailyUsageRate, int daysAhead) {
    if (daysAhead == 0) return available;
    int remaining = available - dailyUsageRate;
    if (remaining < 0) remaining = 0;
    return projectedAvailability(remaining, dailyUsageRate, daysAhead - 1);
}

/* -----
8. POINTERS: pointer-based safe allocation validation
----- */
int validateAllocation(Equipment *e, int qty) {
    if (e == NULL) return 0;
    if (qty <= 0 || qty > e->availableUnits) return 0;
    if (e->availableUnits - qty < e->minReserve) return 0; /* reserve rule */
    return 1;
}

int executeAllocation(Equipment *e, int qty, const char *toLab) {
    if (!validateAllocation(e, qty)) return 0;
    e->availableUnits -= qty;
    e->usageCount += 1;
    char buf[112];
    snprintf(buf, sizeof(buf), "id=%d qty=%d toLab=%s", e->id, qty, toLab);
    logAudit("ALLOCATE", buf);
    return 1;
}

/* -----
9. MULTI-LAB MERGE + CONFLICT DETECTION
----- */
typedef struct {
    int idA, idB;
    char reason[80];
} MergeConflict;

int mergeLabData(Equipment incoming[], int incomingCount,
                 MergeConflict conflicts[], int *conflictCount) {
    int merged = 0;
    *conflictCount = 0;
    for (int i = 0; i < incomingCount; i++) {
        int idx = findEquipmentIndexById(incoming[i].id);
        if (idx == -1) {
            if (addEquipment(incoming[i])) merged++;
        } else {
            if (strcmp(equipment[idx].name, incoming[i].name) != 0 ||
                strcmp(equipment[idx].category, incoming[i].category) != 0) {
                conflicts[*conflictCount].idA = equipment[idx].id;
                conflicts[*conflictCount].idB = incoming[i].id;
                snprintf(conflicts[*conflictCount].reason, 80,
                        "Identity mismatch for id %d (name/category differ)",
                        incoming[i].id);
                (*conflictCount)++;
            } else {
                equipment[idx].quantity += incoming[i].quantity;
                equipment[idx].availableUnits += incoming[i].availableUnits;
                merged++;
            }
        }
    }
}

```

```

    }
    char buf[64];
    snprintf(buf, sizeof(buf), "merged=%d conflicts=%d", merged, *conflictCount);
    logAudit("MERGE", buf);
    return merged;
}

/* -----
10. SMART ALLOCATION ENGINE (explainable, rule-based)
----- */
void recommendAllocation(int equipmentId, int requestedQty, const char *toLab) {
    int idx = findEquipmentIndexById(equipmentId);
    if (idx == -1) { printf("Equipment not found.\n"); return; }
    Equipment e = equipment[idx];

    int maxSafe = e.availableUnits - e.minReserve;
    if (maxSafe < 0) maxSafe = 0;
    int allocation = (requestedQty < maxSafe) ? requestedQty : maxSafe;
    int risk = computeLabRiskScore(e.lab);

    printf("\n--- Allocation Recommendation ---\n");
    printf("Equipment   : %s (ID %d) @ %s\n", e.name, e.id, e.lab);
    printf("Shortage    : %d | Class: %s\n", computeShortageScore(e), classifyEquipment(e));
    printf("Lab Risk    : %d\n", risk);
    printf("Requested   : %d | Safely allocatable: %d\n", requestedQty, maxSafe);
    printf("RECOMMENDATION: allocate %d unit(s) to %s.\n", allocation, toLab);
    printf("REASON: %s has %d/%d units available against a reserve floor of %d; ",
        e.name, e.availableUnits, e.quantity, e.minReserve);
    printf("shortage score %d and lab risk score %d place this request at %s priority.\n",
        computeShortageScore(e), risk,
        (allocation == requestedQty ? "full" : "partial"));
}

/* -----
11. MAINTENANCE / PROCUREMENT PROJECTION (uses recursion above)
----- */
void projectMaintenanceNeed(int equipmentId, int dailyUsageRate, int daysAhead) {
    int idx = findEquipmentIndexById(equipmentId);
    if (idx == -1) { printf("Equipment not found.\n"); return; }
    Equipment e = equipment[idx];
    int projected = projectedAvailability(e.availableUnits, dailyUsageRate, daysAhead);

    printf("\n--- Availability Projection ---\n");
    printf("Equipment: %s | Current available: %d | Daily usage: %d\n",
        e.name, e.availableUnits, dailyUsageRate);
    printf("Projected available after %d day(s): %d\n", daysAhead, projected);
    printf("Outcome: %s\n",
        (projected <= e.minReserve ? "PROCUREMENT/MAINTENANCE ALERT" : "SUPPLY SUSTAINS"));
}

/* -----
12. FILE HANDLING: save / load persistent equipment state
----- */
int saveToFile(void) {
    FILE *fp = fopen(DATA_FILE, "wb");
    if (!fp) return 0;
    fwrite(&equipmentCount, sizeof(int), 1, fp);
    fwrite(equipment, sizeof(Equipment), equipmentCount, fp);
    fclose(fp);
    logAudit("SAVE", DATA_FILE);
    return 1;
}

int loadFromFile(void) {
    FILE *fp = fopen(DATA_FILE, "rb");
    if (!fp) return 0;
    fread(&equipmentCount, sizeof(int), 1, fp);
    fread(equipment, sizeof(Equipment), equipmentCount, fp);
    fclose(fp);
    logAudit("LOAD", DATA_FILE);
    return 1;
}

```



```

/* -----
13. CONSOLIDATED REPORT
----- */

void generateConsolidatedReport(void) {
    printf("\n===== LEMAS-C CONSOLIDATED INVENTORY REPORT =====\n");
    printf("%-4s %-14s %-12s %6s %6s %-16s %-8s\n",
        "ID", "Name", "Lab", "Avail", "Total", "Class", "Score");
    int totalAvailable = 0, mostUsedIdx = -1, worstLabRiskLab = -1;
    for (int i = 0; i < equipmentCount; i++) {
        Equipment e = equipment[i];
        printf("%-4d %-14s %-12s %6d %6d %-16s %-8d\n",
            e.id, e.name, e.lab, e.availableUnits, e.quantity,
            classifyEquipment(e), computeShortageScore(e));
        totalAvailable += e.availableUnits;
        if (mostUsedIdx == -1 || e.usageCount > equipment[mostUsedIdx].usageCount)
            mostUsedIdx = i;
    }
    printf("-----\n");
    printf("Total available units across all labs : %d\n", totalAvailable);
    if (mostUsedIdx != -1)
        printf("Most frequently used equipment      : %s (usage=%d)\n",
            equipment[mostUsedIdx].name, equipment[mostUsedIdx].usageCount);
    (void) worstLabRiskLab;
    printf("===== \n");
    logAudit("REPORT", "consolidated report generated");
}

/* -----
MAIN: demonstration driver (menu-driven in the full submission build)
----- */

int main(void) {
    strcpy(labProfiles[labCount].lab, "CSE-Lab1");
    labProfiles[labCount].totalEquipmentTypes = 0;
    labProfiles[labCount].pendingRequests = 2;
    labCount++;

    Equipment e1 = {201, "Desktop-i5", "Computer", "CSE-Lab1", 40, 6, "Working", 120, 8, 40};
    Equipment e2 = {202, "Arduino-Uno", "Sensor Kit", "CSE-Lab1", 60, 55, "Working", 30, 10, 20};
    Equipment e3 = {203, "HD-Projector", "Projector", "CSE-Lab1", 5, 1, "Under Repair", 80, 1,
200};

    addEquipment(e1);
    addEquipment(e2);
    addEquipment(e3);
    printf("Duplicate add attempt result: %d (expect 0)\n", addEquipment(e1));

    searchByName("Arduino");

    sortByShortageDesc(equipment, equipmentCount);
    generateConsolidatedReport();

    recommendAllocation(201, 5, "CSE-Lab2");
    projectMaintenanceNeed(201, 2, 10);

    Equipment incoming[1] = {{204, "Multimeter", "Instrument", "CSE-Lab1", 15, 12, "Working", 10, 3,
15}};
    MergeConflict conflicts[10];
    int conflictCount = 0;
    mergeLabData(incoming, 1, conflicts, &conflictCount);
    printf("\nMerge conflicts detected: %d\n", conflictCount);

    saveToFile();
    equipmentCount = 0;
    loadFromFile();
    printf("\nReloaded equipment count from file: %d\n", equipmentCount);
    generateConsolidatedReport();

    return 0;
}

```

Appendix B — Test Cases

ID	Feature	Input	Expected Result	Actual Result	Status
T01	Add valid equipment	id=205, qty=20, avail=20	Equipment added, count+1	TO BE FILLED AFTER EXECUTION	PENDING
T02	Invalid quantity rejected	availableUnits > quantity	Add rejected, count unchanged	TO BE FILLED AFTER EXECUTION	PENDING
T03	Duplicate ID rejected	id=201 (existing)	Add rejected	0 returned (verified by source run)	PASS*
T04	Search by name substring	"Arduino"	Matching record(s) listed	1 match printed (verified by source run)	PASS*
T05	Sort by shortage score (desc)	3+ equipment records	Highest shortage score first	TO BE FILLED AFTER EXECUTION	PENDING
T06	Equipment classification	condition=Damaged	Class = CRITICAL	TO BE FILLED AFTER EXECUTION	PENDING
T07	Multi-lab merge (no conflict)	new id, same lab pattern	Record merged, conflicts=0	1 merged / 0 conflicts (verified by source run)	PASS*
T08	Merge conflict detection	existing id, different name	Conflict recorded	TO BE FILLED AFTER EXECUTION	PENDING
T09	Safe allocation (within reserve)	qty below reserve breach	Allocation executed	TO BE FILLED AFTER EXECUTION	PENDING
T10	Unsafe allocation blocked	qty breaches minReserve	Allocation rejected	0 returned; recommendation shows partial/zero allocation (verified by source run)	PASS*
T11	Availability projection	10-day horizon, usage=2/day	Correct projected units returned	Projected 0 units remaining, alert raised (verified by source run)	PASS*
T12	Save to file	4 equipment records in memory	File written successfully	TO BE FILLED AFTER EXECUTION	PENDING
T13	Reload from file	saved file present	Same equipment count restored	4 restored after merge (verified by source run)	PASS*
T14	Consolidated report	4 equipment records	Report lists class, score, totals	Report printed with totals and most-used item (verified by source run)	PASS*

Appendix C — Execution Screenshot Placeholders

Consolidated placeholder index. Populate each item below with the actual compiler/terminal screenshot before final submission; do not substitute fabricated images.

- C.1 Program launch — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.2 Equipment addition — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.3 Input validation — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.4 Duplicate ID detection — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.5 Equipment update — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.6 Search — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.7 Sorting — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.8 Equipment classification — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.9 Recursive analysis — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.10 Multi-lab merge — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.11 Merge conflict detection — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.12 Smart allocation — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.13 Unsafe allocation restriction — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.14 Availability projection — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.15 File save — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.16 File reload — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.17 Consolidated report — [INSERT ACTUAL COMPILER SCREENSHOT HERE]
- C.18 Final integrated execution — [INSERT ACTUAL COMPILER SCREENSHOT HERE]

Appendix D — Sample Data

id	name	category	lab	quantity	availableUnits	condition	usageCount	minReserve	daysSinceMaintenance
201	Desktop-i5	Computer	CSE-Lab1	40	6	Working	120	8	40
202	Arduino-Uno	Sensor Kit	CSE-Lab1	60	55	Working	30	10	20
203	HD-Projector	Projector	CSE-Lab1	5	1	Under Repair	80	1	200
204	Multimeter	Instrument	CSE-Lab1	15	12	Working	10	3	15